SecureConvert — Zero-Trust Ephemeral File Converter

Submitted to Prof. Armando Ruggeri — 2026-09-09

Submitted to: Prof. Armando Ruggeri
Student: Kheizaran Nazari Khakeshoori — kheizarannazarikhakeshoori@gmail.com
Repository: github.com/kheizaran-nazari-khakeshoori/zero-trust-ephemeral-converter
Branch: local_commit — Date: 2026-09-09
Course: Secure Web Systems / Database Systems

Executive Summary

SecureConvert is a small but complete authenticated file-conversion platform built on a zero-trust principle: every file is inspected by magic bytes, processed strictly in RAM, streamed back, and never written to disk. Authentication is two-step (password + TOTP) bound to server-side sessions stored as hashed JWTs in SQLite. The project demonstrates relational design (users, sessions, conversion_jobs, audit_logs), parameterized queries, transactions, migrations, and comprehensive testing.

This report documents architecture, database design, security, API and how to run the system. Accompanying file `docs/db-schema.md` contains the printable ER diagram.

1. Objectives

- Provide ephemeral conversion for text/markdown!'HTML/PDF, JSON!'CSV, PNG!'WebP without persistence.
- Enforce strict file-type validation, size limits and input sanitization.
- Require authenticated access for every conversion, with auditable history per user.
- Show DB best practice: foreign keys, UNIQUE, indexes, hashed secrets, migrations.

2. System Architecture

Stack: Node 22 + Express 4 + SQLite3 + Sharp + PDFKit + Speakeasy (TOTP) + bcryptjs + JWT.

Layout:

```
client/  index.html + app.js  (single page, drag & drop, history)
server/  server.js (Express), config.js (dotenv), authRoutes.js, authMiddleware.js,
         userDb.js (SQLite), converter.js, fileValidator.js, inputValidation.js
docs/    db-schema.md, schema.html, Report_SecureConvert.md
```

Request flow:

```
Browser (127.0.0.1:5500) --CORS--> Express (127.0.0.1:5000) --helmet + rateLimit-->
  /api/auth/register -> bcrypt -> users + audit_logs
  /api/auth/login-step1 -> tempToken -> users.step3Token
  /api/auth/login-step2 -> verify TOTP -> JWT + sessions (tokenHash)
  /api/convert (Bearer + multer.memoryStorage + validateMagicBytes) -> converter -> audit + conversion_jobs -> stream blob
  /api/history (Bearer) -> conversion_jobs
  /api/health -> {status, uptime}
```

All conversions use `multer.memoryStorage()` — buffers never touch disk. PDFs are built with `pdfkit` in a Promise stream, WebP via `sharp({failOnError:true, limitInputPixels:25M})`.

3. Database Design — The Core (Prof. Ruggeri Focus)

3.1 ER

```
users 1 — '  sessions   (FK userId ON DELETE CASCADE)
users 1 — '  conversion_jobs (FK userId ON DELETE CASCADE)
users 1 — '  audit_logs  (FK userId ON DELETE SET NULL — keep trail)
```

See `docs/schema.html` (printable light theme). Mermaid source is in `docs/db-schema.md:7`.

3.2 Tables (from server/userDb.js)

users `id PK, email UNIQUE NOT NULL, passwordHash, mfaSecret, mfaEnabled, step3Token`
- UNIQUE at DB prevents race duplicates beyond JS check. Index `idx_users_email` serves `WHERE email=?`.

sessions `id PK (jti), userId FK, tokenHash UNIQUE, expiresAt, createdAt`
- `tokenHash = SHA256(JWT)` — leaked DB "` stolen session. Index `expiresAt` makes `WHERE expiresAt>now` and `DELETE ... <= now` fast.

conversion_jobs `id PK, userId FK, originalFilename, sourceType, targetType, status, createdAt`
- Indexed `(userId)` + `(createdAt)` for `ORDER BY createdAt DESC LIMIT ?` with `safeLimit 1..100`.

audit_logs `id PK, userId FK, action, ipAddress, createdAt`
- `action` in {register_success, login_step1_success, login_step2_failed_bad_totp, convert_txt_to_html...}, `SET NULL` keeps audit after user deletion.

Pragmas: `PRAGMA foreign_keys=ON`, `PRAGMA journal_mode=WAL` (readers concurrent with writer), `PRAGMA user_version=2` for migrations.

3.3 Migrations

Not just `CREATE TABLE IF NOT EXISTS`. `userDb.js:10` defines `MIGRATIONS[0]=base + [1]=audit_logs+indexes` and `runMigrations()` applies them sequentially based on `user_version`. Re-running is idempotent.

3.4 Integrity Mechanisms

- Parameterized `?` everywhere; `updateUser` whitelists `ALLOWED_USER_COLUMNS` to block column injection.
- `UNIQUE(email)` and `UNIQUE(tokenHash)` are DB invariants.
- `transaction()` (`BEGIN IMMEDIATE / COMMIT / ROLLBACK`) makes `createUser + updateUser(mfaSecret)` atomic.
- `LIMIT` sanitized to 1..100/200 to avoid DoS.

3.5 Key Queries (demo live with `npm run seed`)

-- Q1 login
SELECT * FROM users WHERE email = ?;

-- Q2 history
SELECT id,originalFilename,sourceType,targetType,createdAt FROM conversion_jobs WHERE userId=? ORDER BY createdAt DESC LIMIT ?;

-- Q3 cleanup
DELETE FROM sessions WHERE expiresAt <= ?;

-- Q4 audit
SELECT id,action,ipAddress,createdAt FROM audit_logs WHERE userId=? ORDER BY createdAt DESC LIMIT ?;

-- Q5 professor JOIN
SELECT u.email, COUNT(j.id) AS conversions FROM users u LEFT JOIN conversion_jobs j ON j.userId=u.id GROUP BY u.id ORDER BY conversions DESC;
SELECT targetType, COUNT(*) FROM conversion_jobs GROUP BY targetType;

Try `EXPLAIN QUERY PLAN` before/after the indexes to show logarithmic search.

3.6 Seed

`server/seed.js:1` + `npm run seed` inserts 3 demo users (ada/lin/grace@example.com) with 7 jobs. No `sqlite3` CLI needed on Fedora. Works via `UserDB` API.

---

4. Security Architecture

- **2FA:** password (bcrypt 12) + TOTP (speakeasy, window 1 = ±30s). `mfaSecret` stored server-side; `step3Token` is 32-byte random bridging step1!'step2.
- **JWT:** `jsonwebtoken` 1h expiry, `jti = sessionId (UUID)`, verified in `authMiddleware.js:5` requiring both `payload.jti` and `tokenHash` match.
- **Headers:** `helmet({contentSecurityPolicy:false})`, `x-powered-by` disabled, `X-Content-Type-Options: nosniff`, CORS from `config.js: CORS_ORIGINS`.
- **Rate limiting:** global `RATE_LIMIT_MAX 100/15min` + stricter `AUTH_RATE_LIMIT_MAX 20` on `/api/auth`.
- **File validation:** `fileValidator.js:11` checks PNG/JPG/PDF magic bytes, rejects null bytes/BOM, JSON `JSON.parse`, text heuristic on 1k chars; `inputValidation.js:8` blocks `..`, `/`, `\`, control chars, validates email/password/TOTP regex.
- **Limits:** `MAX_UPLOAD_SIZE` (10 MB), PDF text 2M char cap, JSON 10k rows / 100 headers, PNG 25M pixels.
- **Audit:** every register/login/convert writes `audit_logs` with IP.

---

5. File Conversion Pipelines

- `convertMarkdownToHtml` — escapes HTML, handles `#`, `**`, `*`, `` ` ``, lists !' safe HTML shell.
- `convertMarkdownToPdf` — pdfkit stream, headings sized 24/18/14.
- `convertPngToWebp` — sharp `webp({quality:85})`.
- `convertJsonToCsv` — discovers headers across rows, escapes `"`, `JSON.stringify` for objects, rejects empty/single-type arrays.

Errors return 400 with clear messages; 413 for oversize; 401 on session loss handled on client.

---

6. Frontend (Web GUI)

`client/index.html` — two cards: 3-Step Auth (register, step1, step2, logout) + SecureConvert (drag&drop, format select, progress, recent conversions). `client/app.js:22` derives `API_BASE` from `<meta api-base>` or `localStorage.apiBase` or `127.0.0.1:5000` when served from `:5500`. `npm run dev` (`client/dev-serve.js:1`) runs `serve . -l 5500` and auto-opens `xdg-open/open`. All error handling fixes Blob parsing bug.

---

7. API Specification (excerpt)

- `POST /api/auth/register {email,password} -> 201 {mfaSecret, otpauthUrl}`
- `POST /api/auth/login-step1 {email,password} -> {tempToken}`
- `POST /api/auth/login-step2 {email,tempToken,totpCode} -> {accessToken}`
- `POST /api/convert (Bearer, multipart file+targetFormat) -> blob (html/pdf/csv/webp)`
- `GET  /api/history (Bearer) -> {jobs[]}`
- `GET  /api/health -> {status, uptime, timestamp}`

See `server/server.js` for validation matrix: html/pdf requires `txt/json`, csv requires `json`, webp requires `png`.

---

8. Testing & Quality

cd server
npm test   # 15 tests: converter, fileValidator, auth flow, all 4 conversions, unsupported format, oversize, binary, XSS escaping
npm run lint # 0 errors, _next args ignored
npm audit => 0 vulnerabilities (qs override 6.16.0)

Integration tests spin `http.createServer(app)` on random port, register a fresh `test-${Date.now()}@example.com`, generate TOTP via speakeasy, then hit `/api/convert`.

---

9. Run Instructions

cp server/.env.example server/.env.local   # edit JWT_SECRET: node -e "console.log(require('crypto').randomBytes(48).toString('hex'))"
cd server && npm install && npm start   # http://127.0.0.1:5000
cd client && npm install && npm run dev   # http://127.0.0.1:5500 auto-opens

or: python3 -m http.server 5500

inspect:

Submitted to Prof. Armando Ruggeri — 2026-09-09

npm run seed --prefix server
npm run db:inspect --prefix server
xdg-open docs/schema.html

---

10. Future Work

• Passkey/WebAuthn as 3rd factor (README mentions but not implemented), client-side AES-GCM for storage, per-user rate limit by token, pagination on history, `ETag` for PDF caching, CI (GitHub Actions test+lint).

Appendix — How to Show the Professor (30 sec)

cat docs/db-schema.md
xdg-open docs/schema.html # Print -> Save as PDF
npm run seed --prefix server
curl -s http://127.0.0.1:5000/api/health

Vault is ephemeral: no file ever hits `uploads/` or `temp/` — `multer.memoryStorage()` + streaming.

Prepared for Prof. Armando Ruggeri — September 2026.